

Mock Midterm Exam 1 (Solution Hints)

(6:35 PM – 7:50 PM : 75 Minutes)

- This exam will account for 20% of your overall grade.
- There are five (5) questions, worth 75 points in total. Please answer all of them.
- This is a *closed book, closed notes* exam. *No cheat sheets* are allowed.
- You are allowed to *use scratch papers* for your calculations.

GOOD LUCK!

Question	Parts	Points
1. Asymptotic Bounds	$(a), (b), (c)$	$5 + 5 + 5 = 15$
2. Recurrences.	$(a), (b), (c)$	$5 + 5 + 5 = 15$
3. Cutting Rod	–	12
4. Insertion Sort	–	18
5. Prefix Sums	$(a), (b)$	$10 + 5 = 15$
Total		75

QUESTION 1. [15 Points] Asymptotic Bounds. For each of the following statements state whether it is *True* or *False*. You must justify each answer. Assume that all log's are of base 2.

(a) [5 Points] $2^{\log n^2} = \Theta(n^2)$

(b) [5 Points] $n^{1.2} = \Omega(n \log^2 n)$

(c) [5 Points] $2^{2n} = \mathcal{O}(3^n)$

SOLUTION HINTS.

(a) We have, $2^{\log n^2} = 2^{2 \log n} = (2^{\log n})^2 = n^2 = \Theta(n^2)$

Hence, the given statement is *True*.

(b) We have,

$$\begin{aligned}
 \lim_{x \rightarrow \infty} \left(\frac{n^{1.2}}{n \log^2 n} \right) &= \lim_{x \rightarrow \infty} \left(\frac{n^{0.2}}{\log^2 n} \right) \\
 &= (\ln 2)^2 \lim_{x \rightarrow \infty} \left(\frac{n^{0.2}}{\ln^2 n} \right) && \{ \because \log_2 n = \frac{\ln n}{\ln 2} \} \\
 &= (\ln 2)^2 \lim_{x \rightarrow \infty} \left(\frac{0.2 n^{0.2-1}}{\frac{2 \ln n}{n}} \right) && \{ \text{L'Hopital's rule} \} \\
 &= 0.1 (\ln 2)^2 \lim_{x \rightarrow \infty} \left(\frac{n^{0.2}}{\ln n} \right) \\
 &= 0.1 (\ln 2)^2 \lim_{x \rightarrow \infty} \left(\frac{0.2 n^{0.2-1}}{\frac{1}{n}} \right) && \{ \text{L'Hopital's rule} \} \\
 &= 0.02 (\ln 2)^2 \lim_{x \rightarrow \infty} n^{0.2} \\
 &= \infty \geq c && \{ \text{where, } c \text{ is any positive constant} \}
 \end{aligned}$$

So, $n^{1.2} = \Omega(n \log^2 n)$.

Hence, the given statement is *True*.

(c) We have, $\lim_{x \rightarrow \infty} \left(\frac{2^{2n}}{3^n} \right) = \lim_{x \rightarrow \infty} \left(\frac{4^n}{3^n} \right) = \lim_{x \rightarrow \infty} \left(\left(\frac{4}{3} \right)^n \right) = \infty$.

So, $2^{2n} = \omega(3^n)$.

Hence, the given statement is *False*.

QUESTION 2. [15 Points] Recurrences. For each of the functions below either solve it using the Master Theorem or state why the Master Theorem does not apply.

(a) [5 Points] $\frac{1}{4}T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 4T(\frac{n}{4}) + n^2, & \text{otherwise.} \end{cases}$

(b) [5 Points] $T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 3T(\frac{n}{2}) + (-1)^n n^2, & \text{otherwise.} \end{cases}$

(c) [5 Points] $T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 5T(\frac{n}{3}) + \Theta(n), & \text{otherwise.} \end{cases}$

SOLUTION HINTS.

(a) Multiplying both sides by 4 and observing that $4 \times \Theta(1) = \Theta(1)$ and $4n^2 = \Theta(n^2)$, we have

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 16T(\frac{n}{4}) + \Theta(n^2), & \text{otherwise.} \end{cases}$$

Here, $a = 16$, $b = 4$, $f(n) = \Theta(n^2)$, and $n^{\log_b a} = n^{\log_4 16} = n^2 \Rightarrow f(n) = \Theta(n^{\log_b a} \log^0 n)$.

Hence, using case 2 of the Master Theorem, $T(n) = \Theta(n^{\log_b a} \log^1 n) = \Theta(n^2 \log n)$.

(b) Here, $f(n) = (-1)^n n^2 < 0$ for all odd values of n . Hence, the Master Theorem does not apply in this case.

(c) Here, $a = 5$, $b = 3$, $f(n) = \Theta(n)$, and $n^{\log_b a} = n^{\log_3 5} \Rightarrow f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$ for constant $\epsilon = \log_3(\frac{5}{3}) > 0$.

Hence, using case 1 of the Master Theorem, $T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_3 5})$.

QUESTION 3. [12 Points] Cutting Rod. What the following algorithm EXTENDED-CUT-ROD($p[1..n]$, n) does is not important, but if you must know it does the following. Given a rod of length n inches and a list of prices $p[1..n]$, where $p[i]$ is the selling price of a rod of length i inches, it determines the maximum revenue $r[n]$ obtainable by cutting up the rod and selling the pieces.

```

EXTENDED-CUT-ROD (  $p[1..n]$ ,  $n$  )
1.  $r[0..n] \leftarrow$  new array,  $s[0..n] \leftarrow$  new array
2.  $r[0] \leftarrow 0$ 
3. for  $j \leftarrow 1$  to  $n$  do
4.    $q \leftarrow -\infty$ 
5.   for  $i \leftarrow 1$  to  $j$  do
6.     if  $q < p[i] + r[j - i]$  then
7.        $q \leftarrow p[i] + r[j - i]$ 
8.        $s[j] \leftarrow i$ 
9.    $r[j] \leftarrow q$ 
10. return  $r$  and  $s$ 

```

Analyze the worst-case running time of EXTENDED-CUT-ROD on an input of size n .

SOLUTION HINTS.

Lines 1 and 10 take $\mathcal{O}(n)$ time each and line 2 takes $\Theta(1)$ time.

The outer **for** loop in lines 3–9 iterates n times (from $j = 1$ to $j = n$). In each iteration of that loop lines 4 and 9 take $\Theta(1)$ time. In the j -th iteration of the outer loop the inner **for** loop iterates j times (from $i = 1$ to $i = j$). In each iteration of the inner loop lines 6–8 take $\Theta(1)$ time.

Hence, the overall running time = $\mathcal{O}(n) + \Theta(1) + \sum_{j=1}^n \left(\Theta(1) + \sum_{i=1}^j \Theta(1) \right) = \mathcal{O}(n) + \Theta(1) + \sum_{j=1}^n \left(\Theta(1) + \Theta\left(\sum_{i=1}^j 1\right) \right) = \mathcal{O}(n) + \sum_{j=1}^n (\Theta(1) + \Theta(j)) = \mathcal{O}(n) + \sum_{j=1}^n \Theta(j) = \mathcal{O}(n) + \Theta\left(\sum_{j=1}^n j\right) = \mathcal{O}(n) + \Theta\left(\frac{n(n+1)}{2}\right) = \mathcal{O}(n) + \Theta(n^2) = \Theta(n^2)$.

{I have not explicitly accounted for the last iterations of the loops where the termination condition is satisfied and the loop body is not entered. The reason is: the $\Theta(1)$ cost of those termination iterations is subsumed by the costs of the other steps just outside those loops.}

QUESTION 4. [18 Points] Insertion Sort. Given an array $A[1..n]$ of n distinct numbers as input the *Insertion Sort* algorithm rearranges the items of A in nondecreasing order of value from left (smaller array index) to right (larger array index). Recall that the outermost **for** loop of the algorithm iterates from $j = 2$ to $j = A.length$.

Suppose the following array $A[1..10]$ is given as input to the algorithm. Please show the state of this array after each iteration of the outermost **for** loop.

	1	2	3	4	5	6	7	8	9	10
A	12	8	5	10	15	2	9	18	13	4

There must be one line for each value of j . Please start each line with the j value followed by a “.” and then write down the items (separated by commas, no fancy boxes are needed) of the input array in the order they will appear in $A[1..10]$ at the end of the j -th iteration of the loop.

SOLUTION HINTS.

$j = 2 :$ 8, 12, 5, 10, 15, 2, 9, 18, 13, 4

$j = 3 :$ 5, 8, 12, 10, 15, 2, 9, 18, 13, 4

$j = 4 :$ 5, 8, 10, 12, 15, 2, 9, 18, 13, 4

$j = 5 :$ 5, 8, 10, 12, 15, 2, 9, 18, 13, 4

$j = 6 :$ 2, 5, 8, 10, 12, 15, 9, 18, 13, 4

$j = 7 :$ 2, 5, 8, 9, 10, 12, 15, 18, 13, 4

$j = 8 :$ 2, 5, 8, 9, 10, 12, 15, 18, 13, 4

$j = 9 :$ 2, 5, 8, 9, 10, 12, 13, 15, 18, 4

$j = 10 :$ 2, 4, 5, 8, 9, 10, 12, 13, 15, 18

QUESTION 5. [15 Points] Prefix Sums. The PREFIX-SUM algorithm given below accepts an array $A[p..r]$ of real numbers and updates $S[p : r]$ with prefix sums, that is, for $p \leq i \leq r$, sets $S[i] = \sum_{j=p}^i A[j]$. In order to compute the prefix sums of array $A[1..n]$, PREFIX-SUMS is called with $p = 1$ and $r = n$.

PREFIX-SUM (A, p, r, S)

1. **if** $p = r$
2. $S[p] = A[p]$
3. **else**
4. // split $A[p..r]$ into two approximately equal halves $A[p..q]$ and $A[q + 1..r]$
5. $q = \left\lfloor \frac{p+r}{2} \right\rfloor$
6. // recursively find prefix sums of the left half
7. **PREFIX-SUM (A, p, q, S)**
8. // recursively find prefix sums of the right half
9. **PREFIX-SUM (A, q + 1, r, S)**
10. // update prefix sums of right half with the total sum of the left half
11. **for** $i = q + 1$ **to** r
12. $S[i] = S[i] + S[q]$

- (a) [10 Points] Write a recurrence relation describing the running time of PREFIX-SUM on an array of length n . Assume for simplicity that $n = 2^k$ for some integer $k \geq 0$.
- (b) [5 Points] Solve the recurrence relation from part (a).

SOLUTION HINTS.

- (a) When $n = 1$, line 2 is executed which takes $\Theta(1)$ time. Otherwise, lines 4–12 are executed. Line 5 takes $\Theta(1)$ time and lines 11–12 take $\Theta(n)$ time, and thus taking $\Theta(n)$ time in total. The algorithm calls itself recursively in line 7 on an input of size $\frac{n}{2}$, and similarly in line 9. Let $T(n)$ be the running time of the algorithm on an input of size n . Then

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 2T\left(\frac{n}{2}\right) + \Theta(n), & \text{otherwise.} \end{cases}$$

- (b) Here, $a = 2$, $b = 2$, $f(n) = \Theta(n)$, and $n^{\log_b a} = n^{\log_2 2} = n \Rightarrow f(n) = \Theta(n^{\log_b a} \log^0 n)$. Hence, using case 2 of the Master Theorem, $T(n) = \Theta(n^{\log_b a} \log^1 n) = \Theta(n \log n)$.